H&M Recommendation System

Model Comparison Report

MBA Baseline | ALS Collaborative Filtering | LightGBM Learning-to-Rank

Project: H&M Fashion Retail Personalization Engine

Dataset: H&M Kaggle Competition (~31.7M transactions, 1.37M customers, ~105K articles)

Evaluation: MAP@12 & Hit Rate@12 on 239,951 test-period customers (final 28-day holdout)

Executive Summary

This report introduces and compares three recommendation models forming a progressive hierarchy from association-based rules to full personalized Learning-to-Rank. Each model is presented with its pipeline architecture, technical decisions, and offline evaluation results.

Performance Overview

#	Model	Category	MAP@12	Hit Rate@12	Lift vs Baseline
1	MBA Baseline (FP-Growth)	Association Rules	0.003427	4.32%	1.00x (baseline)
2	ALS Collaborative Filtering	Matrix Factorization	0.011692	7.81%	3.41x
3	LightGBM Learning-to-Rank	Gradient Boosting	0.013956	9.33%	4.07x

Key Takeaway: LightGBM achieves MAP@12=0.013956, a 4.07x uplift over the baseline and 19.4% above ALS

Model 1: MBA Baseline

Market Basket Analysis & FP-Growth + Demographics

1.1 Introduction

The MBA Baseline is the non-personalized benchmark. It extracts cross-selling patterns from historical transactions using the FP-Growth algorithm and handles cold-start users via age-bucketed popularity recommendations. A key innovation is the ECR (Expected Cross-Sell Revenue) metric that prescriptively prioritizes high-margin bundles for VIP customers ("Mature Champions", "Young Whales").

1.2 Pipeline

Step 1: Dual-Window Temporal Filtering

Active Catalog: last 30 days (proxy for live inventory)
Historical Context: last 90 days restricted to active items

Step 2: Native Polars Mathematical Pruning (C++ backend)

min_support = 0.1% | eliminated >3GB RAM crash from mlxtend densification
1,240,764 historical baskets | 146,460 multi-item baskets | 362 active items

Step 3: FP-Growth Frequent Itemset Mining

539 frequent itemsets found | confidence >= 0.20 | lift > 1.0

Step 4: N-to-1 Association Rule Extraction

186 enriched rules exported to Association_Rules.parquet
Antecedent = multi-item cart subset | Consequent = single item

Step 5: Recommendation Engine

Cold-start: age buckets (<25, 25-35, 36-50, >50) -> last-30-day trending
Warm: cart-based rule matching via set(antecedents).issubset(cart)
VIP: ECR = Lift x Confidence x Median_Price | Fallback pad to K=12

1.3 Key Technical Decisions

FP-Growth over Apriori:

3.8x faster on sparse data - compressed FP-Tree vs. iterative generation

Dual-Window Strategy:

Prevents dead-stock recommendations (fast-fashion ~90-day shelf-life)

Native Polars Pruning:

Bypasses mlxtend dense-matrix OOM; runs on local hardware safely

Repurchase tolerance:

Repurchase is a dominant signal in H&M (basics/underwear categories)

ECR for VIP segments:

Prevents recommending cheap items to high-value customers

1.4 Performance

Eval time: 7.3 seconds

Model 2: ALS Collaborative Filtering

Alternating Least Squares - Hu, Koren & Volinsky (2008)

2.1 Introduction

The ALS model transitions to true 1-to-1 personalization. By factorizing the implicit User-Item confidence matrix, the model learns latent preference vectors for every user. Objective: min X,Y sum_ui c_ui(p_ui - x_u^T y_i)^2 + lambda(||x_u||^2 + ||y_i||^2)

2.2 Implicit Confidence Signal

c_ui = sum(quantity_ui * time_weight_ui)   where time_weight decays older transactions (half-life decay).

Matrix scale: 1,338,785 users x 100,959 items | ~26.3M non-zero interactions | Sparsity: 99.98%

2.3 Pipeline

Step 1: Confidence Matrix Construction

quantity * time_weight aggregated per (user, item)
Stored as scipy CSR sparse matrix | Build time: 5.7s

Step 2: ALS Model Training (implicit C++ backend)

Factors=200 | Regularization=0.02 | Alpha=60 | Iterations=25
User factors: (1,338,785 x 200) | Item factors: (100,959 x 200) | 88.6s

Step 3: Memory-Efficient Batch Prediction

Chunk size: 2,000 users | argpartition O(N) top-K extraction
Warm users (216,475): ALS dot product | Cold users (23,476): global popularity

Step 4: Offline Evaluation

Ground truth: test_processed.parquet (239,951 users)
Separate MAP@12 tracked for warm vs. cold cohorts

2.4 Key Technical Decisions

Implicit feedback:

Retail data only shows purchases, not dislikes - requires confidence weighting

time_weight decay:

Penalizes discontinued SKUs from past seasons

200 latent factors:

Balances expressiveness vs. overfitting at 1.3M user scale

C++ backend:

~10x faster than pure NumPy ALS (NumPy fallback kept for reproducibility)

argpartition:

O(N) partial sort vs. O(N log N) - critical for 100K items per user

2K-user chunks:

Prevents OOM from dense 1.3M x 100K matrix materialization

2.5 Performance

Cohort	Users	MAP@12	Hit Rate@12
Overall	239,951	0.011692	7.81% (18,745 users)
Warm users	216,475	0.012522	-
Cold users	23,476	0.004040	-

Lift over Baseline: 3.41x | Cold-start gap: warm MAP@12 is 3.1x higher than cold users

Model 3: LightGBM Learning-to-Rank

Gradient Boosting Binary Classifier for Top-K Retrieval

3.1 Introduction

LightGBM reframes recommendation as supervised binary classification: given a (user, item) candidate pair, predict the purchase probability. A two-stage pipeline - offline feature engineering then fast LightGBM inference - enables 21 multi-dimensional features including semantic Word2Vec similarity.

3.2 Stage 1 - Candidate Generation (05_feature_engineering.py)

Candidate Generation - 3 strategies, capped at 100/user

Repurchase: items user already bought in training window
Popularity: Top-50 globally purchased items (last 7 days)
Item-CF: batched cosine similarity on item-user sparse matrix (last 6 weeks)

Feature Engineering - 21 features per (user, item) pair

User-Item: ui_purchase_count, ui_days_since_last, ui_tw_1w/1m/1s
Item: item_price_mean/max/min, item_total_sales, item_age_mean/max/min
User: user_total_purchases, user_unique_items, user_avg_price, user_active_days, age
Semantic: w2v_cosine_sim (Word2Vec 64-dim, window=5, min_count=3, epochs=5)
Cumulative: cum_purchase_count, cum_avg_price (inactive users)

Label Assignment & Downsampling

Train label=1: bought in last 7 days of train period
Test label=1: bought in test_processed (final 28 days)
Negatives downsampled to 1,500,000 | Train: (1,515,896 x 24) | Test: (21,825,652 x 24)

3.3 Stage 2 - LightGBM Training & Inference

LightGBM Binary Classifier Training

Objective: binary | Metric: AUC | Learning rate: 0.01 | num_leaves: 255
feature_fraction: 0.8 | bagging_fraction: 0.8 | Early stopping: 100 rounds
Best iteration: 840 | Best val AUC: 0.8596 | Training time: 84.9s

Scoring & Top-K Extraction

Score all 21.8M test candidates | Sort per user | Extract Top-12

3.4 Top-10 Feature Importance (by Gain)

Rank	Feature	Description	Type
1	item_total_sales	Total historical units sold	Item popularity
2	w2v_cosine_sim	Word2Vec user-item semantic similarity	Semantic
3	ui_tw_1s	Time-weighted interaction (90 days)	Interaction
4	user_avg_price	User's average purchase price	User profile
5	item_age_mean	Mean age of item buyers	Item profile
6	age	User's age	User profile
7	item_price_mean	Item's mean price	Item profile
8	ui_days_since_last	Days since last purchase of item	Recency
9	user_total_purchases	User's total purchase count	User activity
10	item_price_min	Item's minimum historical price	Item profile

3.5 Performance

Lift vs Baseline: 4.07x | Lift vs ALS

Model Comparison & Key Findings

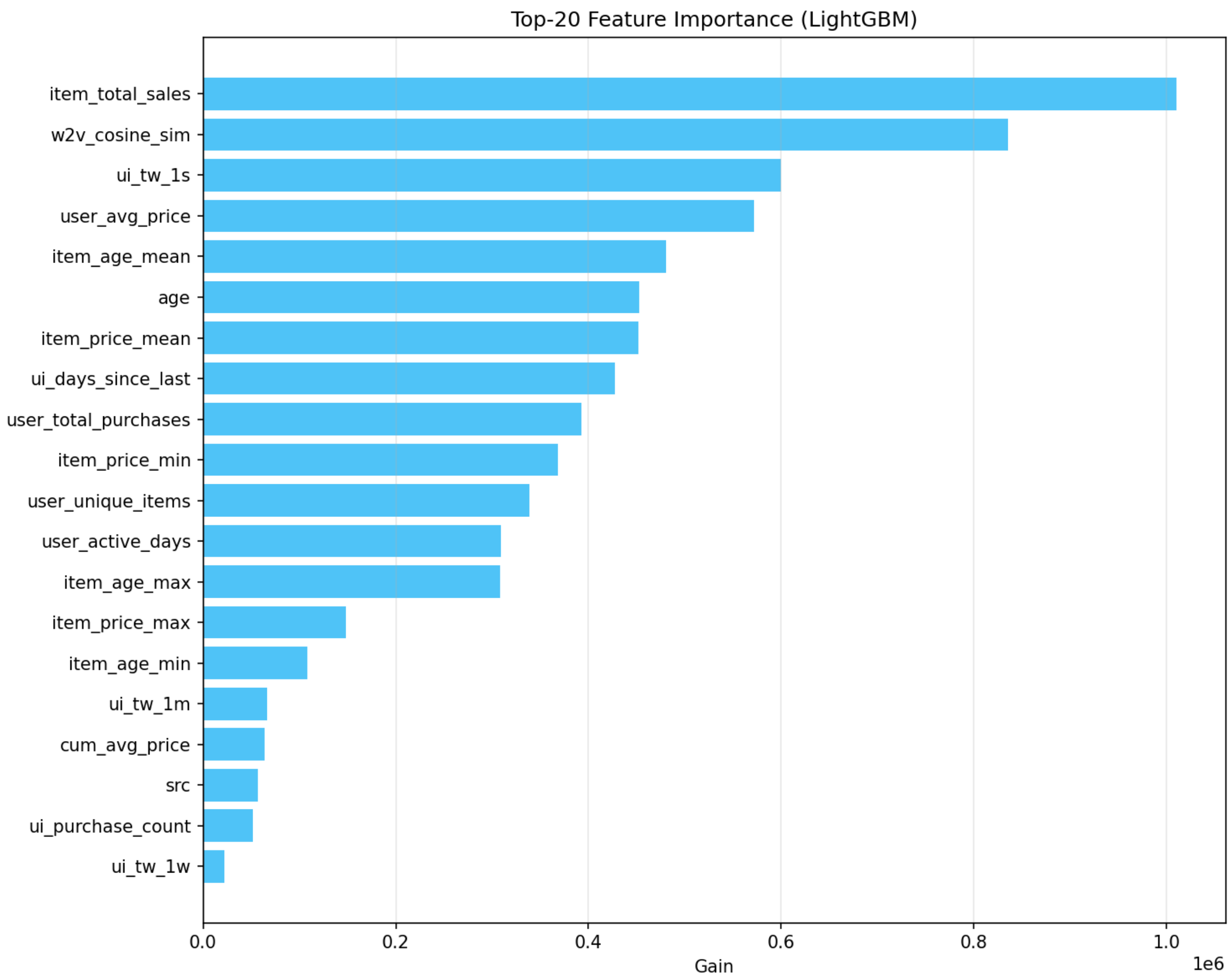
4.1 Head-to-Head Comparison

Dimension	MBA Baseline	ALS	LightGBM
Personalization	Demographic-only	Per-user latent factors	Per-user multi-feature
Cold-start	Age-bucketed trending	Global popularity	Popularity + features
Interpretability	High (Lift/Confidence)	Low (latent space)	Medium (feat importance)
Feature richness	Statistical rules	Interaction history	21 multi-dim features
Business prescriptive	High (ECR, VIP logic)	None	Low (pure ranking)
Inference speed	Very fast (O(1))	Medium (matmul)	Fast (pre-built features)
Training time	~seconds	88.6s	84.9s
MAP@12	0.003427	0.011692	0.013956
Hit Rate@12	4.32%	7.81%	9.33%

4.2 Key Findings

* Personalization drives the biggest gain: ALS delivers 3.41x MAP@12 uplift over the non-personalized baseline.
* Rich features further close the gap: LightGBM's 21 features push MAP@12 19.4% beyond ALS.
* Word2Vec semantic similarity is the 2nd most important feature - purchase sequences contain rich item signals.
* Cold-start remains a bottleneck: ALS warm MAP@12 (0.012522) is 3.1x higher than cold (0.004040).
* MBA Baseline still adds business value: explainable rules + ECR prescriptive VIP logic unavailable in ML models.

4.3 Feature Importance Chart (LightGBM)



Future Improvements & Conclusion

5.1 Future Improvements

Improvement	Expected Impact	Complexity
Two-Tower Neural Network	Higher MAP via deep feature interactions	High
LightGBM LambdaRank	Direct MAP optimization vs. AUC proxy	Medium
Sequential models (GRU4Rec)	Capture purchase sequence dynamics	High
Content-based cold-start	Close warm/cold MAP gap (currently 3.1x)	Medium
Ensemble ALS + LightGBM	Combine latent factors + rich features	Low
Online / real-time learning	React to intra-session signals	Very High

6. Conclusion

The three-model progression demonstrates a clear and measurable improvement hierarchy for the H&M recommendation system, evaluated on an identical 239,951-user holdout set:

* MBA Baseline (MAP@12: 0.003427) - Establishes a rules-based, explainable benchmark with ECR-driven prescriptive VIP business logic.
* ALS Collaborative Filtering (MAP@12: 0.011692) - Introduces true personalization via implicit matrix factorization, delivering a 3.41x improvement.
* LightGBM Learning-to-Rank (MAP@12: 0.013956) - Achieves best offline performance through rich multi-dimensional features and gradient boosting, reaching 4.07x over baseline.

Results confirm that personalization and feature richness are the primary drivers of recommendation quality in H&M's fashion retail context. The warm/cold user gap (3.1x) highlights cold-start resolution as the highest-priority area for future work.

Report generated from: notebooks/03_MBA_Baseline.ipynb | notebooks/4_RecSys_ALS.ipynb | notebooks/05_LightGBM_RecSys.ipynb | scripts/05_feature_engineering.py